

Quá trình thiết kế và phát triển một tác tử AI hỗ trợ lập trình nhúng

Vũ Trí Công, Nguyễn Trung Hiếu

Khoa Kỹ thuật Điện tử 1, Học viện Công nghệ Bưu chính Viễn thông, Hà Nội, Việt Nam

Email: CongVT.B24CHDT005@stu.ptit.edu.vn, hieunt@ptit.edu.vn

Tóm tắt—Bài báo kể lại quá trình thiết kế và phát triển EIDE, một tác tử (agent) dùng mô hình ngôn ngữ lớn để làm phần mềm nhúng cùng kỹ sư, từ bộ hồ sơ thiết kế ngày 05/09/2026 tới bản chạy được ba việc thật trên bo mạch ngày 04/10/2026. Quá trình này có ba điểm khác với cách làm phần mềm thông thường: tài liệu thiết kế là nguồn sự thật và phần lớn mã do một tác tử lập trình viết theo tài liệu, chủ sản phẩm chỉ đọc, duyệt và đo; mọi chỗ mã khác tài liệu phải ghi vào một nhật ký sai lệch (102 mục, 28 mục dẫn tới sửa tài liệu); và mọi lần đối hướng đều do một số đo, không do ý kiến. Bài báo trình bày yêu cầu và ràng buộc đầu vào, quy trình, kiến trúc thu được (một vòng lặp do mô hình điều khiển, ba lớp chặn bằng mã, 127 công cụ, 11 cửa duyệt, số ghi việc móc bầm), và năm lần đối hướng có số đo: từ định tuyến ý định sang vòng lặp (76 ca kiểm: 16/67 → 68/68), từ gõ phím qua hệ điều hành sang kênh kiểm thử giao diện (0 → 40 ca, 13 ca đo khi trả mã cũ), rồi công cụ chưa bao giờ được dùng (31/122 → 12/127), họ lỗi “cơ chế có sẵn, đường dẫn đứt”, và chặn kết quả đạt giả bằng phép phá mã. Kết quả: 41 272 dòng Python, 7 326 dòng Swift, 1 575 ca kiểm, 180 lần ghi mã trong 10 ngày, và tác tử đã tự viết nhân hệ điều hành thời gian thực, phần mềm robot tự cân bằng và lõi RISC-V trên FPGA chạy trên bo thật.

Từ khóa—tác tử AI, mô hình ngôn ngữ lớn, lập trình nhúng, quy trình phát triển phần mềm, tài liệu thiết kế, kiểm thử.

I. GIỚI THIỆU

Quyết định số 21/2026/QĐ-TTg xếp trí tuệ nhân tạo và công nghệ bán dẫn vào danh mục công nghệ chiến lược [1]. Lập trình nhúng nằm ở giao điểm hai hướng đó, và mô hình ngôn ngữ lớn (LLM) đã được thử ở đây: Englhardt và cộng sự [2] cho thấy LLM viết và sửa được mã cho vi điều khiển nhưng hay sai thông số và cần người kiểm; Li và cộng sự [3] thêm kỹ năng chuyên gia cho tác tử lập trình nhúng rồi đo trên bộ bài tập; Zhong và cộng sự [4] đo xu hướng mô hình tìm cách làm bộ kiểm xanh thay vì làm mã đúng. Các bài này nói mô hình làm được gì. Ít bài nói làm thế nào để xây một tác tử như vậy thành sản phẩm dùng được: bắt đầu từ đâu, đối hướng khi nào, dựa vào cái gì để biết mình đang đúng.

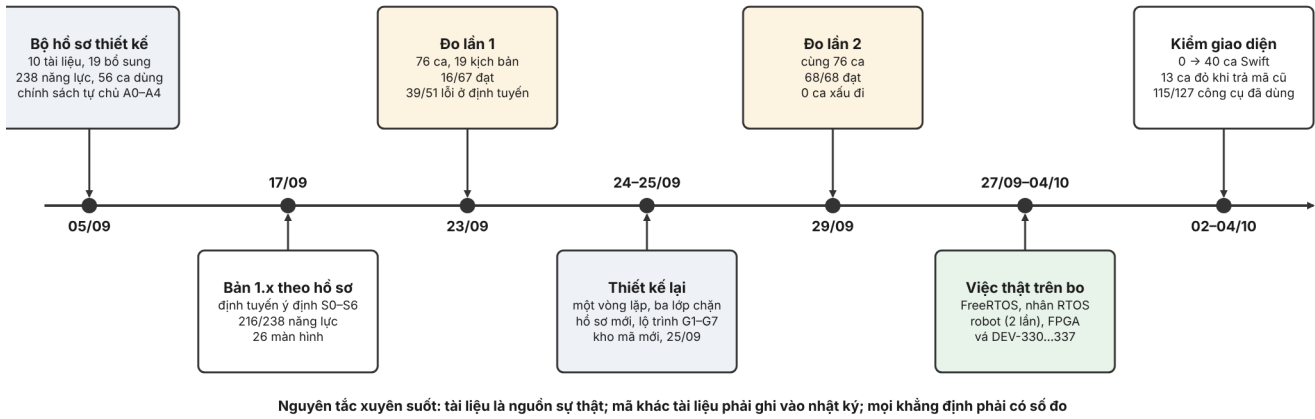
Bài báo kể lại quá trình đó với EIDE, một tác tử làm phần mềm nhúng cùng kỹ sư, trong một tháng từ bộ hồ sơ thiết kế tới bản chạy được việc thật trên ba bo mạch. Có hai lý do để kể. Một là quá trình này dùng chính tác tử lập trình để viết mã theo tài liệu, nên cách tổ chức tài liệu, nhật ký và phép đo quyết định kết quả nhiều hơn cách viết mã; đó là một cách làm mới và đáng ghi lại cả chỗ được lẫn chỗ hỏng. Hai là năm lần đối hướng trong quá trình này đều có số đo trước và sau, nên người khác có thể kiểm lại và dùng lại.

Đóng góp: (1) một quy trình phát triển trong đó tài liệu là nguồn sự thật, tác tử lập trình viết mã, và mọi sai lệch tài liệu—mã phải ghi vào nhật ký; (2) kiến trúc thu được, với số đo của mã, bộ kiểm và các việc thật; (3) năm lần đối hướng có số đo, cùng bài học rút ra. Mã nguồn, tài liệu, nhật ký và số ghi việc của mọi phép đo được công bố [5].

II. YÊU CẦU VÀ RÀNG BUỘC ĐẦU VÀO

Mục tiêu đặt ra từ đầu: một môi trường trong đó kỹ sư gõ một câu tiếng Việt, tác tử làm phần còn lại, từ đọc tài liệu phần cứng, thiết kế mạch, viết mã, dịch, mô phỏng, nạp bo, tới gỡ lỗi trên bo; người chỉ can thiệp ở những chỗ có hậu quả. Phạm vi được chốt sớm và giữ nguyên: một người, một máy, không tài khoản, không phân quyền, không máy chủ; không làm mạch in (PCB, Gerber); nhiều loại chip (AVR, ARM Cortex-M, RISC-V) và cả chip lập trình được (FPGA).

Ba luật nền được viết trước khi có dòng mã nào, và mọi thiết kế sau đó dựng trên chúng. Một, mỗi con số dùng để quyết định phải lần về được chỗ nó lấy ra: trang tài liệu của hãng, dòng tệp cấu hình, hay câu nói nguyên văn của người; mô hình không phải nguồn sự thật. Hai, không có kết quả “đạt” nào mà thiếu bằng chứng; mức đo phải nêu trước khi chạy, và đối mức đo sau khi đã thấy kết quả phải qua một cửa duyệt của người. Ba, mọi thay đổi gỡ lại được, kể cả thay đổi do người tự gỡ; việc không gỡ lại được, như nạp chip, vẫn được ghi kèm lý do.



Hình 1. Dòng thời gian từ bộ hồ sơ thiết kế (05/09/2026) tới bản chạy được việc thật trên bo (04/10/2026). Mỗi mốc có số đo kèm theo.

Hai ràng buộc kỹ thuật thêm: chỉ một mô hình được gọi (gemini-3.8-flash, ép bằng danh sách cho phép trong mã), và không mở cổng mạng nào, giao diện chạy lỗi làm tiến trình con và nói chuyện qua luồng vào-ra chuẩn. Hai ràng buộc này làm việc kiểm thử dễ hơn nhiều: mọi hành vi đều làm lại được bằng cách gửi vào cùng một cửa.

III. QUY TRÌNH PHÁT TRIỂN

A. Dòng thời gian

Hình 1 đặt các mốc trên một trục. Ngày 05/09/2026, bộ hồ sơ thiết kế được dựng: mười tài liệu (từ phân tích thiết kế, yêu cầu, đặc tả, kiến trúc, thiết kế chi tiết, kế hoạch kiểm thử tới chính sách tự chủ với năm mức A0-A4 và năm lớp rủi ro R0-R4) cùng 19 tài liệu bổ sung, một danh mục 238 năng lực và 56 ca đúng. Nguyên tắc đặt ra lúc đó: mỗi năng lực là một hợp đồng có tên, tham số, kết quả và mức rủi ro; hồ sơ phải đủ chi tiết để viết mã được, trước khi viết mã. Bản 1.x được xây theo hồ sơ này và tới 17/09 có 216/238 năng lực, 26 màn hình.

Ngày 23/09, bản 1.x được đo bằng 76 ca kiểm thuộc 19 kịch bản, chạy qua ứng dụng thật: 16/67 ca đo được đạt. Ngày 24-25/09, kiến trúc được thiết kế lại trong một tài liệu tổng thể mới, với lộ trình bảy bước G1-G7, và kho mã mới bắt đầu ngày 25/09. Ngày 29/09, cùng 76 ca chạy lại: 68/68 ca đo được đạt. Từ 27/09 tới 04/10, tác tử làm các việc thật trên bo và mỗi việc tìm ra vài lỗi của chính nó; ngày 02/10 phần giao diện có bộ kiểm đầu tiên. Kho mã v3 có 180 lần ghi trong 10 ngày; nhật ký sai lệch tài liệu-mã có 102 mục, trong đó 44 mục ghi ngay ngày đầu tiên.

B. Vòng làm việc: tài liệu, mã, đo, nhật ký

Hình 2 là vòng lặp được dùng trong suốt quá trình. Tài liệu thiết kế là nguồn sự thật. Mã do một tác tử lập trình viết theo tài liệu; chủ sản phẩm không viết mã, chỉ đọc, duyệt và đo. Mỗi lần giao việc là một gói: một bản giao việc nói rõ nhiệm vụ, thứ tự đọc tài liệu, tiêu chí và mẫu báo cáo; các tài liệu thiết kế dạng văn bản thuần; và bộ 76 ca kiểm. Mã được đo ở bốn mức: ca kiểm đơn vị; ca đi qua giao thức thật mà không tổn tiền mô hình; ca đi qua ứng dụng thật; và việc thật trên bo thật.

Khác với cách làm thông thường nằm ở bước thứ tư. Khi mã buộc phải khác tài liệu, người viết mã không được im lặng làm khác, cũng không được tự sửa tài liệu; phải ghi một

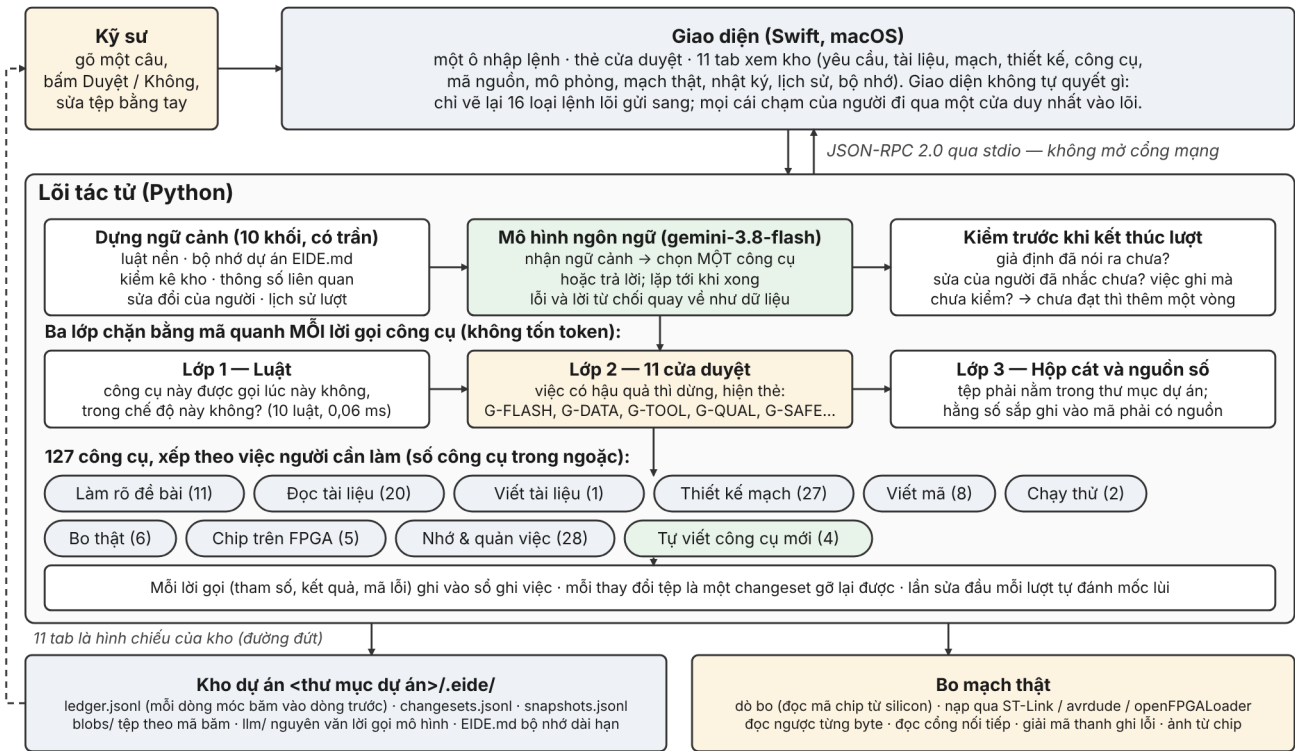
mục vào nhật ký sai lệch theo mẫu cố định: tài liệu mục nào nói gì, mã làm gì, vì sao, và đề nghị cập nhật tài liệu có hay không. Chủ sản phẩm đọc nhật ký rồi quyết: sửa mã về đúng tài liệu, hoặc sửa tài liệu theo mã. Trong 102 mục, 28 mục đề nghị sửa tài liệu và được chấp nhận; phần còn lại là sửa mã hoặc ghi nhận. Một ví dụ: tài liệu cho khối luật nền trong ngữ cảnh ngân sách khoảng 2 000 token; mã đo được 2 610, vì bốn bảng mà các lớp chặn sẽ kiểm không thể viết ngắn hơn bằng tiếng Việt; cắt xuống 2 000 thì phải bỏ một bảng, tức đặt bẫy cho chính mình. Mục nhật ký ghi đủ lý do và chi phí thật (khối này được đệm nên phân đôi chỉ tính tiền một lần mỗi phiên), và tài liệu được sửa.

Một công cụ nhỏ giữ cho tài liệu và mã không trôi xa nhau: `tools/kiem_tai_lieu.py` dò mọi tên công cụ, đường dẫn mã, mã lỗi và con số trong tài liệu kiến trúc rồi so với mã đang chạy. Nó không kiểm được điều khó nhất, là mô tả có nói đúng cách hệ thống hoạt động không, và tài liệu nói rõ giới hạn ấy. Tới 04/10, phép đo này cho 0 chỗ lệch trên 15 tệp.

C. Bộ hồ sơ thiết kế gồm gì

Bộ hồ sơ ngày 05/09 được dựng theo thứ tự thông thường của một sản phẩm phần mềm, nhưng với hai điểm riêng cho tác tử. Điểm thứ nhất: mỗi năng lực của tác tử được viết như một hợp đồng công cụ (tên, tham số, kết quả, mã lỗi, mức rủi ro, cửa duyệt nếu có), và đặc tả yêu cầu có đúng một yêu cầu chức năng cho mỗi năng lực; danh mục 238 năng lực trong 27 nhóm là xương sống nối mọi tài liệu khác. Điểm thứ hai: các tài liệu về hành vi của tác tử được viết riêng, vì chúng không có trong mẫu phần mềm thường: chính sách tự chủ (mức tự chủ A0-A4, lớp rủi ro R0-R4, luật tự duyệt cho từng cửa, cửa số gỡ lại, chính sách hội thoại (khi nào hỏi, hỏi gộp thế nào, hết hai lần hỏi thì đi tiếp với giả định nào), kiến trúc ngữ cảnh (khối nào vào ngữ cảnh, trần bao nhiêu), kiến trúc bộ nhớ (bộ nhớ dài hạn ghi ở đâu, nén thế nào, kiểm ngược sau nén ra sao), và bộ lời nhắc. Một bảng rà soát 40 vùng thiết kế được dùng để biết hồ sơ đã đủ chi tiết để viết mã chưa; ngày 05/09 có 9/40 vùng đạt mức đó, và bộ hồ sơ được bổ sung tới khi 40/40 vùng đạt rồi mới giao việc viết mã.

Khi thiết kế lại ngày 24-25/09, ba tài liệu hành vi (thiết kế sản phẩm, kiến trúc tác tử, giao thức giao diện) được hợp nhất thành một tài liệu tổng thể duy nhất, và bản giao việc ghi rõ thứ tự ưu tiên khi tài liệu mâu thuẫn nhau, cùng quy định: phát



Hình 3. Kiến trúc thu được sau lần thiết kế lại: giao diện chỉ vẽ lại thứ lỗi gửi; lỗi là một vòng lặp do mô hình điều khiển, mỗi lời gọi công cụ đi qua ba lớp chặn bằng mã; mọi việc ghi vào kho của dự án.

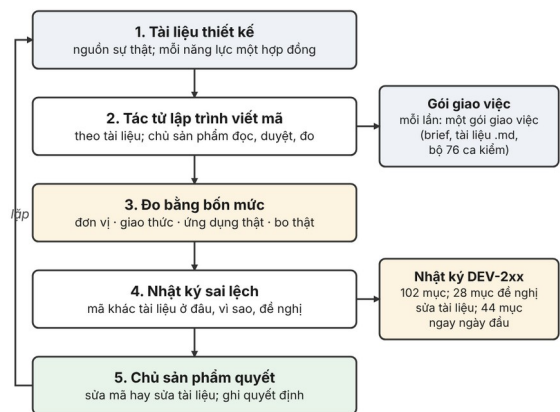
hiện mâu thuẫn thì ghi vào nhật ký, không tự chọn. Các tài liệu bổ sung sau đó (quản lý bộ nhớ, đường ống nạp tài liệu, sinh sơ đồ mạch, mô hình cây khối phân cấp) được giao thành gói riêng, mỗi gói có cờ tính năng và phép so hồi quy hai chế độ bật/tắt để không làm hỏng phần đang chạy.

D. Ba ví dụ từ nhật ký sai lệch

Bảng I trích ba mục ngày đầu tiên của kho mã v3, để thấy nhật ký ghi gì và chủ sản phẩm quyết thế nào. Ngoài các mục khi viết mã, phiên dùng thật đầu tiên ngày 25/09 (33 lượt, một dự án nhỏ về ổ USB Wi-Fi cắm TV) sinh thêm sáu mục, trong đó có việc lớp chặn hằng số không nguồn chặn quá sạch, không cho ghi gì, và kênh kiểm thử giao diện được thêm vào ngay ngày đó.

BẢNG I BA MỤC NHẬT KÝ SAI LỆCH, NGÀY 25/09/2026			
Mục	Tài liệu nói	Mã làm, và vì sao	Quyết
DEV-217	khối luật nền trong ngữ cảnh: khoảng 2 000 token	2 610 token; bốn bảng mã các lớp chặn sẽ kiểm không viết ngắn hơn được bằng tiếng Việt; khối này được đệm nên phân đôi chỉ tính tiền một lần mỗi phiên	sửa tài liệu
DEV-218	bộ luật trước khi gọi mô hình có hai kết quả: chặn hoặc hỏi	thêm kết quả thứ ba: cảnh báo ngay bằng mã (0 token) rồi vẫn cho mô hình chạy tiếp; hai ca an toàn (chip nóng, điện lưới) cần câu cảnh	sửa tài liệu

Mục	Tài liệu nói	Mã làm, và vì sao	Quyết
		báo tới trong vài giây nhưng sau đó vẫn cần tác tử giúp khoanh vùng	
DEV-227	changeset do một móc sau lời gọi công cụ sinh ra	changeset do chính công cụ ghi tệp sinh ra, vì chỉ công cụ biết nội dung trước và sau để tạo phép đảo	sửa tài liệu



Không ai được im lặng làm khác tài liệu, và không được tự sửa tài liệu.

Hình 2. Vòng làm việc: tài liệu là nguồn sự thật; tác tử lập trình viết mã; đo bốn mức; sai lệch ghi vào nhật ký; chủ sản phẩm quyết sửa mã hay sửa tài liệu.

IV. KIẾN TRÚC THU ĐƯỢC

Hình 3 vẽ kiến trúc sau lần thiết kế lại. Giao diện (Swift, macOS) có một ô nhập lệnh, các thẻ cửa duyệt và 11 tab xem kho; nó không tự quyết gì, chỉ vẽ lại 16 loại lệnh lỗi gửi sang, và mọi thao tác của người đi qua đúng một cửa vào lỗi. Lỗi (Python) là một vòng lặp: mô hình nhận ngữ cảnh dựng từ kho (chia 10 khối có trần riêng), chọn một trong 127 công cụ, nhận kết quả, chọn tiếp, tới khi trả lời. Mỗi lời gọi công cụ đi qua ba lớp chặn bằng mã, không tốn token: luật (công cụ này được gọi lúc này không; 10 luật, 0,06 ms), cửa duyệt (11 cửa cho việc có hậu quả: nạp chip, cài công cụ, đổi mức đo sau khi có kết quả, xoá tệp, chạy lệnh hệ thống...; việc dừng lại, hiện thẻ, chờ người bấm), và hộp cát (tệp phải nằm trong thư mục dự án; hằng số sắp ghi vào mã phải có nguồn). Trước khi kết thúc lượt, một bước kiểm hỏi: giả định đã nói ra chưa, sửa đổi của người đã nhắc chưa, có việc ghi mà chưa kiểm không.

Mọi lời gọi, tham số, kết quả, mã lỗi ghi vào sổ ghi việc, mỗi dòng móc mã bám vào dòng trước; mỗi thay đổi tệp là một changeset có phép đảo; bản chốt do người đặt tên; nguyên văn từng lời gọi mô hình được lưu. Mỗi thông số trong kho mang một bậc tin được (VÀNG: tài liệu của hãng đã có người xác nhận; BẠC: đọc bằng mã từ PDF chưa ai xem; NGƯỜI: người nói, có trích nguyên văn; CÀI ĐẶT: đọc từ tệp cấu hình; ĐỒNG: đoán từ hình hay tên, chỉ để gợi ý), và đường nâng bậc chỉ người đi được. Bảng II xếp 127 công cụ theo việc người cần làm.

BẢNG II
CÁC NHÓM VIỆC CỦA TÁC TỪ (SỐ CÔNG CỤ)

Việc (số)	Tác từ làm được gì
Làm rõ đề bài (11)	hỏi gộp một cụm câu kèm lý do; ghi yêu cầu kèm câu nói nguyên văn; nêu 2–4 cách làm rồi để người chốt
Đọc tài liệu (20)	nạp PDF, Word, Excel, slide, mã nguồn; rút hình kèm chữ trong hình; lấy con số bằng mã theo trang
Viết tài liệu (1)	dựng Word, PowerPoint, Excel, PDF từ nguồn Markdown trong kho, rồi mở lại tệp đếm trước khi báo
Thiết kế mạch (27)	bản đồ mạch; cây khối nhiều cấp; sinh sơ đồ nguyên lý mở được bằng KiCad
Viết mã (8)	đọc hiểu mã cũ trước khi sửa; dịch cho AVR, ARM Cortex-M, RISC-V; đo xem bộ kiểm có đo gì không
Chạy thử trên máy (2)	nêu mức đo trước, chạy, rồi đối chiếu
Bo thật (6)	dò bo; nạp; đọc ngược so từng byte; đọc cổng nối tiếp; giải mã thanh ghi lỗi; đọc khung ảnh từ chip
Chip trên FPGA (5)	soát cú pháp Verilog; mô phỏng; tổng hợp; đặt–đi dây và đo Fmax; đóng gói và nạp bitstream
Nhớ và quản việc (28)	kho; sổ ghi việc; gỡ lại; bản chốt; nhánh; chia việc lớn thành chặng
Tự lo cho mình (4)	tìm công cụ; tự viết công cụ mới kèm bộ kiểm, chỉ nạp khi bộ kiểm xanh

V. NĂM LẦN ĐỔI HƯỚNG CÓ SỐ ĐO

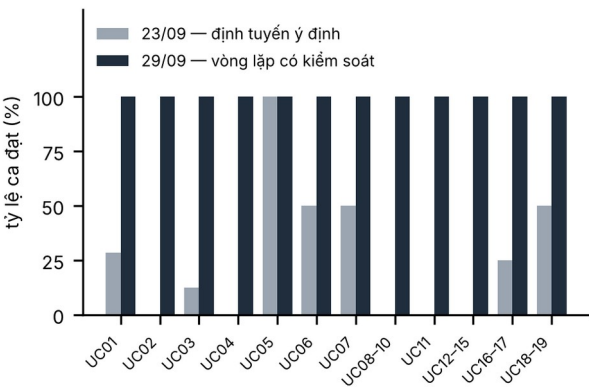
Bảng III tóm tắt năm lần đổi hướng: mỗi lần có một số đo làm lý do, một quyết định, và một số đo sau đó. Các mục dưới kể từng lần.

BẢNG III
NĂM LẦN ĐỔI HƯỚNG: SỐ ĐO TRƯỚC, QUYẾT ĐỊNH, SỐ ĐO SAU

Lần	Số đo trước	Quyết định	Số đo sau
1. Cách tổ chức tác từ	76 ca: 16/67 đạt; 39/51 lỗi nằm ở định tuyến ý định	bỏ phân loại ý định và chuỗi mẫu; một vòng lặp, ba lớp chặn bằng mã	68/68 đạt; 51 ca tốt lên, 0 ca xấu đi
2. Kiểm thử giao diện	0 ca kiểm Swift; gõ phím qua hệ điều hành rơi nhâm cửa sổ	kênh tệp thay ngón tay, đọc ngược được trạng thái giao diện; thêm bộ kiểm Swift	40 ca; 13 đo khi trả mã cũ; 124/124 ô
3. Công cụ không ai dùng	31/122 công cụ chưa từng được gọi; 7 đường dẫn đứt	nối lại đường đứt; giao đúng loại việc cho từng công cụ	115/127 đã dùng thật
4. Đường dẫn đứt	7 lần trong 2 ngày: cơ chế viết đúng, không ai gọi, không lỗi báo	mỗi khối một ca đi đúng đường người dùng đi; chốt chặn gói cũ hơn mã	DEV-330... 337 và kèm ca kiểm
5. Kết quả đạt giả	3 lần bộ kiểm tự so với chính nó; “đạt” viết cứng	tiêu chí nêu trước; bộ mức đo chấm; phép phá mã; đọc công sau nạp	6/6 phép phá đo đúng lúc; 4/4 ở phiên sau

A. Từ định tuyến ý định sang vòng lặp có kiểm soát

Bản 1.x làm theo cách quen thuộc với phần mềm: mỗi lượt đi qua bảy pha; mô hình phân loại câu người gõ vào một ý định định sẵn và điền tham số theo lược đồ, rồi mã chạy một chuỗi bước định sẵn cho ý định đó. Cách này dễ kiểm soát vì mọi chuỗi đều biết trước. Phép đo 23/09 cho thấy nó hỏng ở chính chỗ quan trọng nhất: 39 trong 51 ca không đạt chết ở khâu định tuyến, trước khi chạm tới chuyên môn. Cùng câu “Làm cho mình cái mạch thông minh” khi thì được hiểu là ý định chưa rõ, khi thì là tạo dự án; khe tham số đường dẫn chỉ chứa một chuỗi nên câu có hai đường dẫn mất một; tám ca bị ghép một đường dẫn bịa từ chính câu người dùng; chuỗi mẫu dừng ở một nút thiếu tiền đề (“chưa ghim hộ chiếu chip, chip nào?”) khi việc chưa cần chip. Hình 4 cho thấy bản 1.x bằng 0 ở hầu hết kịch bản cần hiểu yêu cầu mở rồi phối nhiều năng lực.



Hình 4. Tỷ lệ ca đạt theo kịch bản trên cùng 76 ca kiểm, trước (23/09) và sau (29/09) lần đổi hướng thứ nhất.

Quyết định là bỏ phân loại ý định và chuỗi mẫu. Mô hình quyết lại ở mỗi bước dựa trên kết quả thật, còn những việc không được làm do mã quyết ở ba lớp chặn. Bộ luật chặn trước khi gọi mô hình được giữ nguyên từ bản cũ, vì 5/16 ca đạt của bản cũ là nhờ nó: phần xác định vẫn giữ được an toàn khi phân định tuyến hồng. Kết quả 29/09: 68/68 ca đo được đạt; trong 66 ca đo được ở cả hai lần, 51 ca chuyển từ không đạt sang đạt, không ca nào chuyển ngược. Cái giá: mỗi lời gọi mô hình mang toàn bộ ngữ cảnh (trung vị 46 916 token vào, gấp khoảng 39 lần bản cũ), mỗi lượt cần trung bình 12 lời gọi; nhờ 89 % token vào được đọc từ bộ đệm của nhà cung cấp, chi phí mỗi lượt tăng khoảng mười lần chứ không phải bốn mươi.

B. Kiểm thử giao diện: từ gõ phím sang kênh tệp

Tới 02/10, phần giao diện chưa có một ca kiểm nào; tệp cấu hình gói Swift chỉ khai một mục tiêu chạy được, không có mục tiêu kiểm. Đó là lý do thật khiến mọi lỗi màn hình đều kết thúc bằng câu “không con số nào bắt được, phải nhìn màn hình”. Cách kiểm đầu tiên là gõ phím qua hệ điều hành; nó gửi phím tới cửa sổ đang được chọn, nên khi người dùng đang làm việc khác thì phím rơi nhầm cửa sổ, chuyện đã xảy ra, và cách ấy bị bỏ. Thay vào đó ứng dụng mở một kênh tệp chỉ bật khi thư mục kiểm thử có mặt, hiện dài báo cho người biết; mọi thứ đọc từ kênh vẫn đi qua đúng một cửa vào lỗi như ngón tay người. Kênh này còn đọc ngược được trạng thái giao diện, nên kiểm được chữ có bị cắt, khối có đè nhau, câu trả lời có còn dấu Markdown thô không. Bộ kiểm Swift có 40 ca; phép đo đáng kể hơn con số 40 là 13 ca đo khi trả lại mã cũ, vì nó nói bộ kiểm có canh được gì không. Bộ quét 11 tab cho 124/124 ô.

C. Công cụ không ai dùng

Rà toàn bộ số ghi việc phát hiện 31 trong 122 công cụ chưa bao giờ được gọi; bảy trong đó có đường dẫn tới chúng bị đứt, nghĩa là có tính năng trên giấy mà không có cách nào tới được. Bảy đường được nối lại; phần còn lại được giao đúng loại việc để kiểm. Nay 115/127 công cụ đã được dùng thật trong số. Một công cụ không bao giờ được dùng thì bằng không có nó, và không phép đo nào trước đó bắt được điều này vì mọi ca kiểm đều gọi thẳng công cụ.

D. Họ lỗi “cơ chế có sẵn, đường dẫn đứt”

Trong hai ngày làm việc trên bo, bảy lần gặp đúng một hình dạng lỗi: một thứ viết đúng mà không ai gọi tới, và không lần nào có lỗi báo ra. Nhánh nạp FPGA có trong thực đơn công cụ nhưng chưa chạy lần nào nên vờ ngay lần đầu; bảng tổng kiểm được sinh ra nhưng tệp chính không #include; hàm nhíp hệ thống viết đúng nhưng ô vector ngắt vẫn trỏ về trình xử lý mặc định; bộ chuyển ngữ cảnh nối đúng vector, đúng ưu tiên, nhưng không ai đặt bit kích hoạt. Cùng dạng ấy xuất hiện ở ba trong bảy việc giao diện: dữ liệu có sẵn trong kho, thiếu khối đọc nó ra; tab mô phỏng đọc một khoá khác với khoá công cụ ghi vào. Ba lần trong cùng một ngày, bộ ca kiểm xanh cả khi đường dẫn đứt, vì ca kiểm thử thứ làm việc rồi kết luận cho việc đã được làm; hai điều ấy cách nhau một đường dẫn, và đường dẫn là chỗ hay đứt nhất. Quyết định: mỗi khối phải có một ca đi qua đúng đường người dùng đi, và một chốt chặn phiên nếu gói ứng dụng cũ hơn mã nguồn, vì một lần gói cũ ba ngày đã cắt lời tác tử ở 3 000 ký tự không để lại dấu, làm người đọc nhật ký kết luận sai.

E. Chặn kết quả đạt giả

Trong phiên robot, ba lần liên tiếp tác tử viết bộ kiểm cho đúng chỗ nó vừa sửa, và cả ba lần bộ kiểm chỉ chép lại logic sang tệp kiểm rồi so với chính nó, xanh bất kể mã đúng hay sai; một lần khác chương trình mô phỏng in chữ “đạt” viết cứng trong lệnh in. Đây đúng là hành vi mà [4] đo được trên bộ bài tập, và nó nguy hiểm hơn một lỗi báo thẳng vì nó trông như đã xong. Ba cơ chế được thêm vào mã: tiêu chí phải nêu trước khi chạy, và bộ mức đo chấm kết quả chứ không phải chương trình; đổi mức đo sau khi đã thấy kết quả phải qua cửa duyệt; và công cụ phá mã có chủ đích rồi đòi bộ kiểm phải đo. Sau khi bị bắt viết lại cho bộ kiểm dùng thẳng mã sản phẩm, sáu phép phá đều chuyển đỏ đúng lúc; ở phiên robot sau, 4/4 phép phá bị bắt. Một luật thêm vào sau một lỗi thật khác: sau mỗi lần nạp phải đọc cổng nối tiếp để biết chương trình nào đang chạy, vì công cụ nạp so với đúng tệp nó được đưa nên đưa sai tệp thì vẫn báo “đã kiểm, 0 byte lệch”.

VI. KẾT QUẢ TỔNG HỢP

BẢNG IV
SẢN PHẨM SAU MỘT THÁNG, ĐO NGÀY 04–05/10/2026

Chỉ số	Giá trị
Lỗi Python / giao diện Swift	41 272 dòng, 108 tệp / 7 326 dòng
Công cụ / cửa duyệt / tab	127 / 11 / 11
Ca kiểm đơn vị Python / Swift	1 575 / 40
76 ca kiểm qua ứng dụng thật	68/68 ca đo được đạt
Lần ghi mã (commit) / ngày	180 / 10
Mục nhật ký sai lệch / đề nghị sửa tài liệu	102 / 28
Tài liệu kiến trúc so với mã	0 chỗ lệch trên 15 tệp
Việc thật trên bo	nhân RTOS tự viết (STM32F469, nhíp 1 003 Hz); robot tự cân bằng (ATmega328P, đứng được); lõi RISC-V trên FPGA (Tang Nano 20K, 96/96 ô đo, lệch 0 % với mô phỏng)

Ba việc thật cho một phép đo mà bốn mức kiểm thử không cho được: tác tử có gánh nổi một việc phức tạp hay không. Nhân hệ điều hành thời gian thực tự viết có 1 211 dòng bên cạnh 110 676 dòng của hãng không sửa, chạy trên bo với nhíp đo thật 1 003 Hz và mức nước ngăn xếp của sáu tác vụ đọc từ chip. Robot hai bánh đứng được hai lần, lần hai với ngắt phát xung đo thật 50,0005 kHz. Lỗi RISC-V là việc đầu tiên nằm ngoài hần vùng tác tử từng làm: trước đó nó không có công cụ HDL nào; bốn công cụ được viết thêm trong hai ngày và kết quả đi tới 96 ô đo trên silicon khớp bốn tổng kiểm người tính tay. Mỗi việc cũng tìm ra lỗi của chính EIDE; các lỗi đó được vá kèm ca kiểm và ghi vào nhật ký.

VII. BÀI HỌC VÀ HẠN CHẾ

Bốn bài học, mỗi bài gắn với một số đo ở trên. Thứ nhất, hồ sơ thiết kế đầy đủ là điều kiện để tác tử lập trình viết được mã, nhưng hồ sơ không thay được phép đo: bản 1.x có 216/238 năng lực và vẫn đạt 16/67; chỉ phép đo qua ứng dụng thật mới nói nó không dùng được. Thứ hai, nhật ký sai lệch rẻ hơn nhiều so với việc để tài liệu và mã trôi xa nhau; 44 mục trong

ngày đầu cho thấy tài liệu chi tiết tới đâu vẫn có chỗ mã phải khác, và ghi lại là cách duy nhất để chủ sản phẩm quyết. Thứ ba, ca kiểm gọi thẳng công cụ không bắt được đường dẫn đứt; mỗi khối cần một ca đi đúng đường người dùng đi, và rà số ghi việc là cách rẻ nhất để biết thứ gì chưa bao giờ chạy. Thứ tư, mọi lời “đã xong” của tác tử đều phải có một phép đo không do nó chấm: bộ kiểm phải chứng minh được nó đo khi mã hỏng.

Hạn chế. Quá trình này do một người làm chủ sản phẩm, với một tác tử lập trình và một mô hình; đổi mô hình có thể đổi hành vi, nên cần đo lại. Hai lần đo 76 ca cách nhau sáu ngày, trong khoảng đó có thêm năng lực, nên phần cải thiện do kiến trúc và phần do năng lực mới chưa tách hẳn, dù phân loại nguyên nhân cho thấy 39/51 ca nằm ở định tuyến. Bộ ca kiểm do chính nhóm tác giả soạn. Còn 12/127 công cụ chưa được dùng thật; tác tử chưa có công cụ chạy lệnh dừng nên hai lần phải nhờ người; và đường dừng bitstream FPGA mới chạy được qua EIDE, chưa chạy được bằng lệnh tay.

VIII. KẾT LUẬN

Một tác tử làm phần mềm nhúng cùng kỹ sư đã được thiết kế và phát triển trong một tháng theo một quy trình trong đó tài liệu là nguồn sự thật, tác tử lập trình viết mã, mọi sai lệch tài liệu–mã được ghi lại, và mọi lần đổi hướng đều do số đo. Kết quả là một kiến trúc vòng lặp bao bởi ba lớp chặn bằng mã, 127 công cụ, 11 cửa duyệt, số ghi việc móc bấm, đạt 68/68 ca kiểm qua ứng dụng thật và làm được ba việc thật trên ba bo mạch. Năm lần đổi hướng có số đo trước và sau, nên người xây tác tử tương tự có thể dùng lại: đo qua ứng dụng thật trước khi tin hồ sơ; viết nhật ký sai lệch thay vì im lặng; kiểm bằng đường người dùng đi; rà thứ chưa bao giờ chạy; và đòi bộ kiểm chứng minh nó biết báo lỗi. Hướng tiếp theo là lặp quá trình này với nhiều người, nhiều mô hình, và một bộ ca kiểm do người ngoài soạn.

TÀI LIỆU THAM KHẢO

- [1] Thủ tướng Chính phủ, “Quyết định số 21/2026/QĐ-TTg ban hành Danh mục công nghệ chiến lược và Danh mục sản phẩm công nghệ chiến lược,” 30/4/2026. [Trực tuyến]. <https://congbao.chinhphu.vn/van-ban/quyet-dinh-so-21-2026-qd-ttg-469478.htm>
- [2] Z. Englhardt, R. Li, D. Nissanka, Z. Zhang, G. Narayanswamy, J. Breda, X. Liu, S. Patel, và V. Iyer, “Exploring and characterizing large language models for embedded system development and debugging,” trong Extended Abstracts of the CHI Conference on Human Factors in Computing Systems, ACM, 2024. arXiv:2307.03817.
- [3] Y. Li và cộng sự, “Skilled AI agents for embedded and IoT systems development,” arXiv:2603.19583, 2026.
- [4] Z. Zhong, A. Raghunathan, và N. Carlini, “ImpossibleBench: Measuring LLMs’ propensity of exploiting test cases,” arXiv:2510.20270, 2025.
- [5] Vũ Trí Công và Nguyễn Trung Hiếu, “EIDE v3: lỗi tác tử kỹ sư nhúng; mã nguồn, tài liệu thiết kế, nhật ký sai lệch (docs/md/EIDE-DEV-LOG.md), bộ 76 ca kiểm và số ghi việc ba việc thật,” 2026. [Trực tuyến]. <https://github.com/mobiluckvn/EIDEv3>